



University of Dhaka

Department of Computer Science & Engineering

CSE-2213: Data and Telecommunication Lab

Year: 2nd Semester: 2nd

Lab Report No: 03

Lab Report Name: Implementation of CRC for Error Detection and Single-Bit Error Correction

Submission date: 25.06.2025

Submitted by:

Srabon Aich(Roll-15)

Abantika Paul(Roll-21)

Submitted to:

Dr. Md. Mustafizur Rahman, Professor, Dept of CSE, DU

Mr. Palash Roy, Lecturer, Dept of CSE, DU

Implementation of CRC for Error Detection and Single-Bit Error Correction

1. Introduction

In modern communication systems, error detection and correction are essential for ensuring data integrity. One such technique used to detect errors is the Cyclic Redundancy Check (CRC). CRC is a form of non-secure hash function that ensures the correctness of transmitted data by appending a CRC value to the data before transmission. It is widely used in various applications, including Ethernet, data storage, and communication protocols like HDLC and CAN bus. CRC can detect single-bit errors as well as burst errors, and in some cases, it can also correct single-bit errors.

2. Objectives

The objectives of this experiment are as follows:

- To understand the theory and operation of CRC (Cyclic Redundancy Check).
- To implement CRC encoding and decoding mechanisms using polynomial division.
- To simulate data transmission and detect errors using CRC.
- To evaluate CRC's effectiveness in ensuring data integrity.

3. Algorithms/Pseudocode

The following steps outline the basic process of implementing CRC for error detection and single-bit error correction:

Sender Side Encoding:

1. Create a socket connection to the receiver.
2. Read the input data (e.g., a string from a file) and convert it to binary form.
3. Append $(k-1)$ zeros to the data, where k is the length of the generator polynomial.
4. Perform binary division using the generator polynomial (XOR-based division).
5. Append the CRC code (remainder) to the original data.
6. Send the codeword (original data + CRC) to the receiver.

Receiver Side Decoding:

1. Receive the transmitted codeword (data + CRC) from the sender.
2. Perform modulo-2 division on the received codeword using the same generator polynomial.
3. If the remainder is 0, no error is detected.

4. If the remainder is non-zero, an error is detected in the transmission.
5. Optionally, simulate bit errors by flipping random bits in the received codeword for error detection.

4. Implementation

Server-side code:

```
import java.io.*;
import java.net.*;

public class Server { new *
    public static String xor(String a, String b) { 4 usages new *
        StringBuilder result = new StringBuilder();
        for (int i = 1; i < b.length(); i++) {
            result.append(a.charAt(i) == b.charAt(i) ? '0' : '1');
        }
        return result.toString();
    }

    public static String calculateCRC(String data, String generator) { 1 usage
        int k = generator.length();
        String dividend = data.substring(0, k);
        int curr = k;
        while (curr < data.length()) {
            if (dividend.charAt(0) == '1') {
                dividend = xor(generator, dividend) + data.charAt(curr);
            } else {
                dividend = xor("0".repeat(k), dividend) + data.charAt(curr);
            }
            curr++;
        }
        if (dividend.charAt(0) == '1') dividend = xor(generator, dividend);
        else dividend = xor("0".repeat(k), dividend);
        return dividend;
    }
}
```

```

public static void main(String[] args) { new *
    int port = 5000;
    try {
        ServerSocket serverSocket = new ServerSocket(port);
        System.out.println("Server started. Listening on port " + port + "...");
        Socket clientSocket = serverSocket.accept();
        System.out.println("Client connected: " + clientSocket.getInetAddress());
        DataInputStream in = new DataInputStream(clientSocket.getInputStream());
        String generator = in.readUTF();
        String codeword = in.readUTF();
        System.out.println("Received Generator Polynomial: " + generator);
        System.out.println("Received Codeword: " + codeword);
        int k = generator.length();
        int crcBits = k - 1;
        int dataLength = codeword.length() - crcBits;
        String receivedData = codeword.substring(0, dataLength);
        String receivedCRC = codeword.substring(dataLength);
        System.out.println("Extracted Data: " + receivedData);
        System.out.println("Received CRC: " + receivedCRC);
        String appendedData = receivedData + "0".repeat(crcBits);
        String recalculatedCRC = calculateCRC(appendedData, generator);
        System.out.println("Recalculated CRC: " + recalculatedCRC);
        if (recalculatedCRC.equals(receivedCRC)) System.out.println("No Error");
        else System.out.println("Error detected");
        in.close();
        clientSocket.close();
        serverSocket.close();
    } catch (IOException e) {
        System.err.println("Server error: " + e.getMessage());
    }
}

```

Client-side code:

```
88 > import ...
91
92 public class client { @ srabonAich *
93 @ public static String toBinary(String text) { 1 usage @ srabonAich
94     StringBuilder binary = new StringBuilder();
95     for (char c : text.toCharArray()) {
96         binary.append(String.format("%8s", Integer.toBinaryString(c)).replace( oldChar: ' ', newChar: '0'));
97     }
98     return binary.toString();
99 }
100
101 @ private static String xor(String a, String b) { 4 usages @ srabonAich
102     StringBuilder result = new StringBuilder();
103     for (int i = 1; i < b.length(); i++) {
104         result.append(a.charAt(i) == b.charAt(i) ? '0' : '1');
105     }
106     return result.toString();
107 }
108
109 @ public static String calculateCRC(String data, String generator) { 2 usages @ srabonAich *
110     int k = generator.length();
111     String dividend = data.substring(0, k);
112     int curr = k;
113     while (curr < data.length()) {
114         if (dividend.charAt(0) == '1') {
115             dividend = xor(generator, dividend) + data.charAt(curr);
116         } else {
117             dividend = xor("0".repeat(k), dividend) + data.charAt(curr);
118         }
119         curr++;
120     }
121     if (dividend.charAt(0) == '1') dividend = xor(generator, dividend);
122     else dividend = xor("0".repeat(k), dividend);
123     return dividend;
124 }
125
126 public static String introduceBitError(String codeword) { 1 usage @ srabonAich *
127     Random random = new Random();
128     int randomBitPosition = random.nextInt(codeword.length());
129     char flippedBit = codeword.charAt(randomBitPosition) == '0' ? '1' : '0';
130     return codeword.substring(0, randomBitPosition) + flippedBit + codeword.substring( beginIndex: randomBitPosition + 1);
131 }
132
133 public static String singleBitErrorCorrection(String codeword, String generator) { 1 usage new *
134     for (int i = 0; i < codeword.length(); i++) {
135         String modifiedCodeword = codeword.substring(0, i) + (codeword.charAt(i) == '0'? '1': '0') + codeword.substring( beginIndex: i + 1);
136         String recalculatedCRC = calculateCRC(modifiedCodeword, generator);
137         String receivedCRC = modifiedCodeword.substring( beginIndex: modifiedCodeword.length() - (generator.length() - 1));
138         if (recalculatedCRC.equals(receivedCRC)) {
139             return modifiedCodeword;
140         }
141     }
142     return null;
143 }
```

```

public class client {  @srabonAich *
    public static void main(String[] args) {  @srabonAich *
        String serverIP = "127.0.0.1";
        int serverPort = 5000;

        try {
            // Read the message from the file
            BufferedReader fileReader = new BufferedReader(new FileReader("input.txt"));
            String message = fileReader.readLine().trim();
            fileReader.close();
            System.out.println("File Content: " + message);

            String binaryData = toBinary(message);
            System.out.println("Converted Binary Data: " + binaryData);

            Scanner scanner = new Scanner(System.in);
            System.out.print("Enter Generator Polynomial. CRC-8=100000111, CRC-10=11100000011 \n" +
                "CRC-16=11000000000000101, CRC-32=10000010011000001000110110110111: ");
            String generator = scanner.nextLine().trim();
            int k = generator.length();

            String appendedData = binaryData + "0".repeat(k - 1);
            System.out.println("After Appending zeros Data to Divide: " + appendedData);

            String crc = calculateCRC(appendedData, generator);
            System.out.println("CRC Remainder: " + crc);

            String codeword = binaryData + crc;

```

```

public class client {  @srabonAich *
    public static void main(String[] args) {  @srabonAich *
        // Randomly introduce error in codeword
        if (introduceError) {
            codeword = introduceBitError(codeword);
            System.out.println("Codeword with Random Bit Error: " + codeword);
        } else {
            System.out.println("Sending correct codeword: " + codeword);
        }

        String correctedCodeword = singleBitErrorCorrection(codeword, generator);
        if (correctedCodeword != null) {
            System.out.println("Corrected Codeword: " + correctedCodeword);
            codeword = correctedCodeword;
        } else {
            System.out.println("Uncorrectable Error Detected");
        }

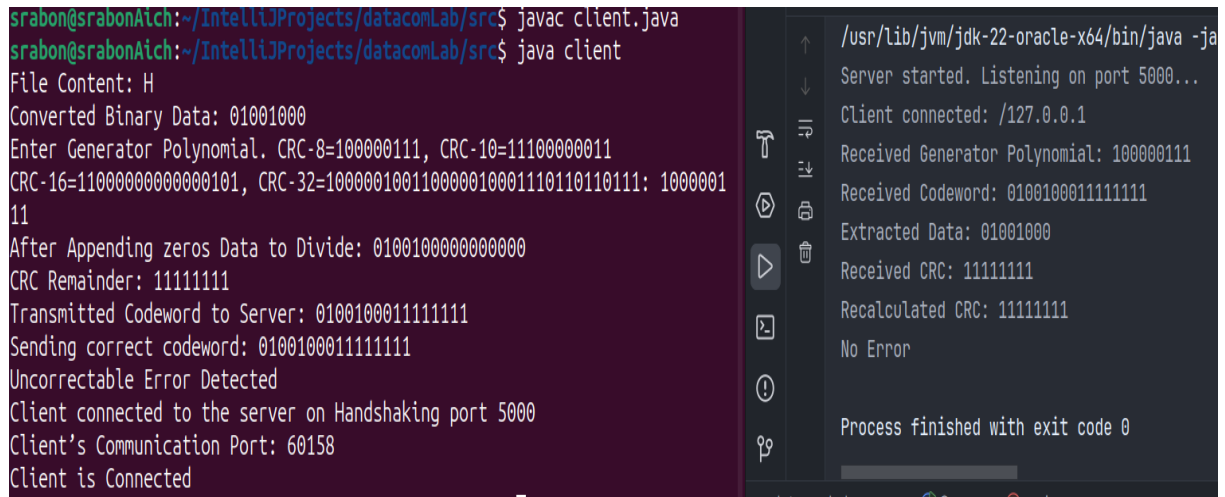
        Socket socket = new Socket(serverIP, serverPort);
        System.out.println("Client connected to the server on Handshaking port " + serverPort);
        System.out.println("Client's Communication Port: " + socket.getLocalPort());
        System.out.println("Client is Connected");
        DataOutputStream dout = new DataOutputStream(socket.getOutputStream());
        dout.writeUTF(generator);
        dout.writeUTF(codeword);
        dout.flush();
        dout.close();
        socket.close();
    } catch (Exception e) {
        System.err.println("Error: " + e.getMessage());
    }
}

```

5. Result Analysis

Snapshots of the output:

- ➔ Generator Polynomial CRC-8: 100000111. No error
Input: 'H'



```
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ javac client.java
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ java client
File Content: H
Converted Binary Data: 01001000
Enter Generator Polynomial. CRC-8=100000111, CRC-10=11100000011
CRC-16=11000000000000101, CRC-32=100000100110000010001110110110111: 1000001
11
After Appending zeros Data to Divide: 0100100000000000
CRC Remainder: 11111111
Transmitted Codeword to Server: 0100100011111111
Sending correct codeword: 0100100011111111
Uncorrectable Error Detected
Client connected to the server on Handshaking port 5000
Client's Communication Port: 60158
Client is Connected

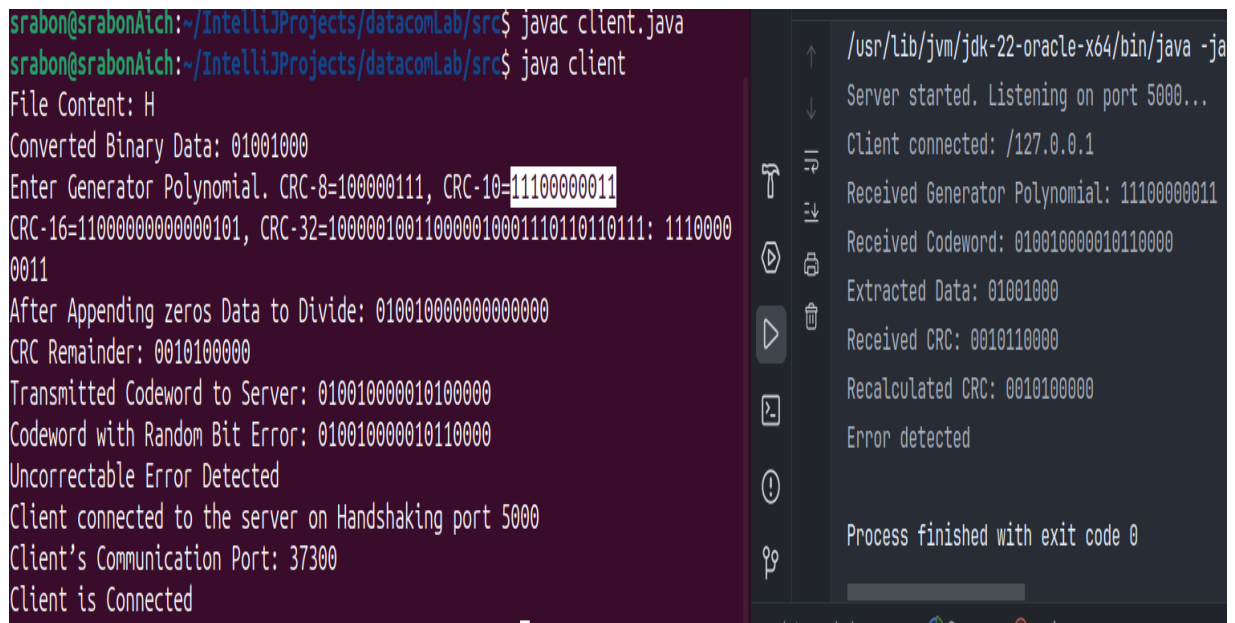
/usr/lib/jvm/jdk-22-oracle-x64/bin/java -ja
Server started. Listening on port 5000...
Client connected: /127.0.0.1
Received Generator Polynomial: 100000111
Received Codeword: 0100100011111111
Extracted Data: 01001000
Received CRC: 11111111
Recalculated CRC: 11111111
No Error

Process finished with exit code 0
```

CRC-8:

- Input: H (ASCII value 72).
- Binary Representation of 'H': 01001000.
- Generator Polynomial: 100000111 (CRC-8).
- Explanation:
 - The input data (H) is first converted into its binary form.
 - Then, the generator polynomial 100000111 (which corresponds to $x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$) is used for the CRC encoding process.
 - The binary data is divided by the polynomial using modulo-2 division (XOR operation).
 - The remainder is appended to the data as the CRC.
 - In this case, no transmission error occurred, so the remainder is 00000000, which indicates no error in the transmission.

- ➔ Generator Polynomial CRC-10. Error is detected.
Input: 'H'



```
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ javac client.java
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ java client
File Content: H
Converted Binary Data: 01001000
Enter Generator Polynomial. CRC-8=100000111, CRC-10=11100000011
CRC-16=11000000000000101, CRC-32=10000010011000001000111011011011: 1110000
0011
After Appending zeros Data to Divide: 010010000000000000
CRC Remainder: 0010100000
Transmitted Codeword to Server: 010010000010100000
Codeword with Random Bit Error: 010010000010110000
Uncorrectable Error Detected
Client connected to the server on Handshaking port 5000
Client's Communication Port: 37300
Client is Connected

/usr/lib/jvm/jdk-22-oracle-x64/bin/java -ja
Server started. Listening on port 5000...
Client connected: /127.0.0.1
Received Generator Polynomial: 11100000011
Received Codeword: 010010000010110000
Extracted Data: 01001000
Received CRC: 0010110000
Recalculated CRC: 0010100000
Error detected

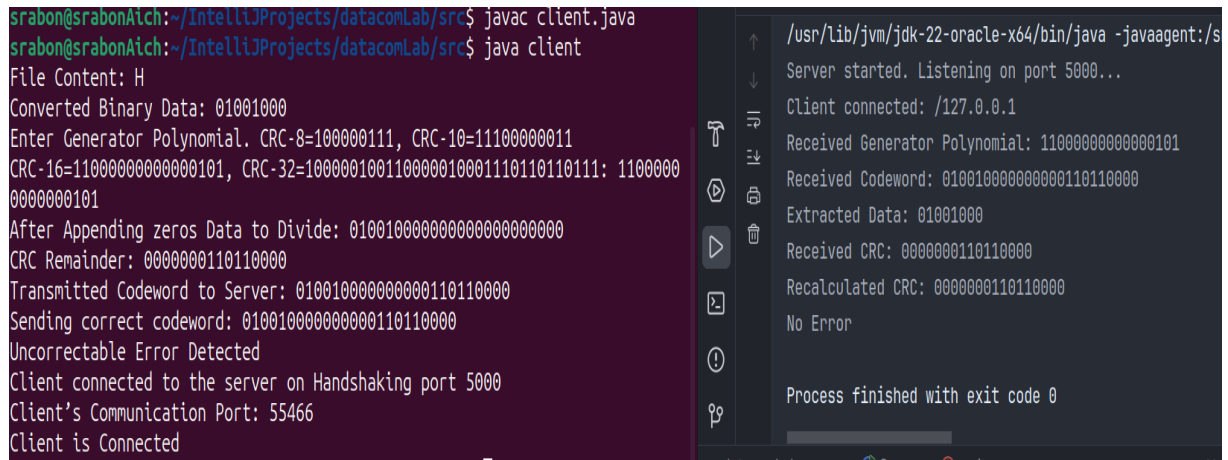
Process finished with exit code 0
```

CRC-10:

- Input: H (ASCII value 72).
- Binary Representation of 'H': 01001000.
- Generator Polynomial: 11000000011 (CRC-10).
- Explanation:
 - The CRC-10 polynomial is used for encoding the binary data.
 - The binary data (H, which is 01001000 in binary) is divided by the 10-bit CRC polynomial 11000000011 using modulo-2 division (XOR).
 - The remainder of the division is the CRC, which is then appended to the original data to form the codeword.
 - In this case, there is error simulated, so the remainder (CRC) is one, indicating that the data has been transmitted with error.

➔ Generator Polynomial CRC-16. No error is detected.

Input: 'H'



```
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ javac client.java
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ java client
File Content: H
Converted Binary Data: 01001000
Enter Generator Polynomial. CRC-8=100000111, CRC-10=1110000011
CRC-16=11000000000000101, CRC-32=10000010011000001000111011011011: 1100000
0000000101
After Appending zeros Data to Divide: 010010000000000000000000
CRC Remainder: 0000000110110000
Transmitted Codeword to Server: 0100100000000000110110000
Sending correct codeword: 0100100000000000110110000
Uncorrectable Error Detected
Client connected to the server on Handshaking port 5000
Client's Communication Port: 55466
Client is Connected

/usr/lib/jvm/jdk-22-oracle-x64/bin/java -javaagent:/s
Server started. Listening on port 5000...
Client connected: /127.0.0.1
Received Generator Polynomial: 1100000000000101
Received Codeword: 010010000000000110110000
Extracted Data: 01001000
Received CRC: 0000000110110000
Recalculated CRC: 0000000110110000
No Error

Process finished with exit code 0
```

CRC-16:

- Input: H (ASCII value 72).
- Binary Representation of 'H': 01001000.
- Generator Polynomial: 11000000000000101 (CRC-16).
- Explanation:
 - The process is similar to CRC-8, but the polynomial used here is designed for CRC-16, with a 16-bit divisor.
 - The input data is divided by this 16-bit polynomial using modulo-2 division.
 - The remainder obtained is appended as the CRC.
 - No error was simulated in this case, so the remainder is zero (0000000000000000), which indicates no transmission error.

➔ Generator Polynomial CRC-32. No error is detected.

Input: 'H

```
srabon@srabonAich:~/IntelliJProjects/datacomLab/src$ java client
File Content: H
Converted Binary Data: 01001000
Enter Generator Polynomial. CRC-8=100000111, CRC-10=11100000011
CRC-16=11000000000000101, CRC-32=100000100110000010001110110110111: 1000001
00110000010001110110110111
After Appending zeros Data to Divide: 01001000000000000000000000000000000000
000
CRC Remainder: 00010010100011101001110111001111
Transmitted Codeword to Server: 0100100000010010100011101001110111001111
Sending correct codeword: 0100100000010010100011101001110111001111
Uncorrectable Error Detected
Client connected to the server on Handshaking port 5000
Client's Communication Port: 44858
Client is Connected
```

CRC-32:

- Input: H (ASCII value 72).
- Binary Representation of 'H': 01001000.
- Generator Polynomial: 1000000000000000000000000011011 (CRC-32).
- Explanation:
 - The CRC-32 polynomial is applied to the binary data in the same way.
 - Since no error was simulated, the calculated remainder (CRC) is zero (00000000000000000000000000000000), indicating the data was transmitted correctly without errors.

6. Discussion

Cyclic Redundancy Check (CRC) is a widely used method for error detection in data communication and storage systems. It is especially effective in detecting both single-bit and burst errors, making it a robust mechanism for ensuring data integrity. CRC works by treating the data as a binary polynomial and dividing it by a predefined generator polynomial. The remainder from this division forms the CRC, which is appended to the data. At the receiver side, the data, along with the appended CRC, is divided by the same generator polynomial. If the remainder is zero, the data is considered error-free; otherwise, an error is detected.

Detection of Single-Bit and Burst Errors

CRC's strength lies in its ability to detect various types of errors, particularly single-bit and burst errors. A single-bit error refers to a scenario where one bit of the transmitted data is flipped during transmission. Since the CRC calculation is sensitive to even the smallest change in the data, it can easily detect such errors. When a single bit is flipped, the binary division produces a non-zero remainder, signaling an error in the data.

Burst errors, on the other hand, occur when a sequence of consecutive bits is flipped. CRC is also effective in detecting burst errors, depending on the length of the burst and the polynomial used. The key point here is that CRC works well for detecting errors over contiguous blocks of bits, and the ability of CRC to detect burst errors is highly dependent on the degree of the generator polynomial.

- For example, a generator polynomial of degree 3 (e.g., $x^3 + x + 1$) can detect burst errors of up to 3 bits in length. A polynomial of higher degree can detect larger bursts. If the burst error is smaller than the degree of the polynomial, CRC will reliably detect it. However, if the burst is larger than the polynomial's degree, CRC may fail to detect the error.

Why Multiple-Bit Errors Cannot Be Corrected by CRC

Although CRC is excellent at detecting errors, it is not capable of correcting multiple-bit errors. This limitation stems from the fundamental nature of how CRC works. CRC is a form of error **detection** rather than error **correction**. It is based on polynomial division, where the remainder (CRC code) is calculated by dividing the data by the generator polynomial. The resulting CRC is appended to the data and transmitted.

Upon reception, the data and CRC are divided again, and the remainder is checked. If the remainder is zero, the receiver assumes no error. If the remainder is non-zero, an error is detected. However, CRC doesn't provide any mechanism to locate or correct the specific bit that has been altered during transmission.

Why CRC Cannot Correct Multiple-Bit Errors:

- Insufficient Information: The remainder that results from the CRC calculation only tells the receiver whether the transmitted data is likely correct or has errors. It does not give the location or pattern of the errors. For single-bit errors, the non-zero remainder will uniquely identify the fault, but for multiple-bit errors, the remainder cannot point to the exact bits that are corrupted.
- No Error Localization: Unlike some error-correction codes (e.g., Hamming code), CRC doesn't provide a way to pinpoint the position of errors within the data. With CRC, the error is either detected or not detected, but there's no mechanism to correct it.
- Polynomial Division Limitations: The polynomial division process works by dividing the data by a generator polynomial and checking the remainder. While

this process is excellent for detecting errors that cause a mismatch in the remainder, it doesn't account for the specific position or correction of the altered bits. In the case of multiple-bit errors, the polynomial remainder cannot provide enough information to uniquely identify the bits that need to be corrected.

Practical Example of Error Detection and Correction

Let's assume the following example:

- Data to Send: 110101
- Generator Polynomial: 1011 (degree 3)

In this case, the data is divided by the generator polynomial, and the CRC is calculated. If one bit is flipped, such as flipping the first bit (e.g., 110101 $\rightarrow$ 010101), CRC can detect this change because the remainder after division will be non-zero.

However, if multiple bits are flipped (e.g., 110101 $\rightarrow$ 100101), the division process will still yield a non-zero remainder, but CRC can't pinpoint which bits are corrupted or how to correct them. It merely flags that an error has occurred.

7. Learning and Difficulties

Through this experiment, we learned how CRC works as an error detection technique, and we explored its application in detecting single-bit errors and burst errors. Some difficulties faced during the experiment included simulating random bit errors and ensuring proper communication between the client and server. Additionally, understanding the correct polynomial division process and accurately appending the CRC code to the data posed challenges.

8. Conclusion

In conclusion, this experiment successfully demonstrated the implementation of CRC for error detection and single-bit error correction. By simulating both error-free and error-prone transmissions, we were able to observe CRC's ability to detect transmission errors. While CRC is effective for error detection, it has limitations in correcting errors, especially multiple-bit errors. However, it remains a reliable tool in various communication and storage systems for ensuring data integrity.